Loan Approval Prediction

Executed project report with saved notebook outputs, tables, and charts

Project Description

Loan Approval Prediction

Project Summary

This project is a clean, student-friendly implementation of a loan approval classification. The main goal is to predict loan approval decisions from applicant and credit details. The notebook keeps the workflow simple: load the dataset, understand the columns, clean the data, train a baseline model, and check how well the model performs.

No external repository links or profile links are included in this description. Everything needed to understand and run the project is kept inside this folder.

Problem Statement

A raw dataset is useful only when it is converted into a clear decision-making workflow. In this project, the data is processed step by step so that important patterns can be found and a machine learning model can be trained. The expected output is based on loanstatus.

Files Included

File	Purpose	Quick Note
loanapprovaldataset.csv	Main data source for this folder	2000 sampled rows x 13 columns
Notebook file	Complete Python workflow	Includes loading, cleaning, training, and evaluation
PDF report	Human-readable project summary	Matches the updated project structure

Important Columns

Column	Role
loanid	Input feature used in the modelling workflow
noofdependents	Input feature used in the modelling workflow
education	Input feature used in the modelling workflow
selfemployed	Input feature used in the modelling workflow
incomeannum	Input feature used in the modelling workflow
loanamount	Input feature used in the modelling workflow
loanterm	Input feature used in the modelling workflow
cibilscore	Input feature used in the modelling workflow
residentialassetsvalue	Input feature used in the modelling workflow
commercialassetsvalue	Input feature used in the modelling workflow
luxuryassetsvalue	Input feature used in the modelling workflow
bankassetvalue	Input feature used in the modelling workflow

Workflow Followed

1. Load the data from the local dataset file present in the same project folder.
2. Inspect the structure using shape, column names, missing values, and basic statistics.
3. Clean the dataset by removing empty columns, handling missing values, and keeping only useful features.
4. Prepare the model input by separating features and the target column.
5. Train a baseline model using a simple scikit-learn pipeline with preprocessing.
6. Evaluate the result using practical metrics such as accuracy, classification report, MAE, RMSE, or R2 score depending on the target type.
7. Write observations in a short and direct way so the project can be explained easily.

How to Run

1. Open the notebook in Jupyter Notebook, JupyterLab, Google Colab, or VS Code.
2. Keep the dataset files in the same folder as the notebook.
3. Run the cells from top to bottom.
4. Read the printed metrics and notes at the end before making conclusions.

Final Note

This version keeps the logic understandable and avoids unnecessary complexity. The aim is not to make the notebook look overloaded, but to make it readable, explainable, and useful for a student-level data science portfolio.

Executed Notebook Outputs

The outputs below were generated from the saved notebook after running the code cells in this project folder.

Cell 2 - 2. Load the dataset

```
Dataset shape: (4269, 13)
```

```
loan_id no_of_dependents education self_employed income_annum \
0 1 2 Graduate No 9620797
1 2 0 Not Graduate Yes 4135595
2 3 2 Graduate No 9133296
3 4 4 Graduate No 8235831
4 5 5 Not Graduate Yes 9830244

loan_amount loan_term cibil_score residential_assets_value \
0 29968333 12 778 2400000
1 12185859 8 414 2700000
2 29619234 21 504 7100000
3 30767180 7 467 18200000
4 24148136 21 385 12400000

commercial_assets_value luxury_assets_value bank_asset_value \
0 17580987 22821601 7960057
1 2143185 8897489 3304093
2 4469237 33236145 12822965
3 3347640 23379530 7921890
4 8201339 29303151 5023144

loan_status
0 Approved
1 Rejected
2 Rejected
3 Rejected
4 Rejected
```

Cell 3 - 3. Understand the data

```
Rows: 4269
Columns: 13
```

```
column dtype missing_values unique_values
0 loan_id int64 0 4269
1 no_of_dependents int64 0 7
2 education object 0 2
3 self_employed object 0 2
4 income_annum int64 0 4269
5 loan_amount int64 0 4268
6 loan_term int64 0 21
7 cibil_score int64 0 605
8 residential_assets_value int64 0 278
9 commercial_assets_value int64 0 4215
10 luxury_assets_value int64 0 4268
11 bank_asset_value int64 0 4266
12 loan_status object 0 2
```

Cell 4 - 4. Clean the data

```
Shape after simple cleaning: (4269, 13)
```

```
loan_id no_of_dependents education self_employed income_annum \
0 1 2 Graduate No 9620797
1 2 0 Not Graduate Yes 4135595
2 3 2 Graduate No 9133296
3 4 4 Graduate No 8235831
4 5 5 Not Graduate Yes 9830244

loan_amount loan_term cibil_score residential_assets_value \
0 29968333 12 778 2400000
1 12185859 8 414 2700000
2 29619234 21 504 7100000
3 30767180 7 467 18200000
4 24148136 21 385 12400000

commercial_assets_value luxury_assets_value bank_asset_value loan_status
0 17580987 22821601 7960057 Approved
1 2143185 8897489 3304093 Rejected
2 4469237 33236145 12822965 Rejected
3 3347640 23379530 7921890 Rejected
4 8201339 29303151 5023144 Rejected
```

Cell 5 - 5. Select the target column

```
Selected target column: loan_status
```

```
0 Approved
1 Rejected
2 Rejected
3 Rejected
4 Rejected
Name: loan_status, dtype: object
```

Cell 6 - 6. Quick EDA

```
Numeric columns: 10
Categorical columns: 3
No missing values found in the previewed columns.
```

```
count mean std min \
loan_id 4269.0 2.135000e+03 1.232498e+03 1.0
no_of_dependents 4269.0 2.552823e+00 1.795635e+00 0.0
income_annum 4269.0 5.059137e+06 2.807016e+06 158171.0
loan_amount 4269.0 1.513505e+07 9.043713e+06 240327.0
loan_term 4269.0 1.089225e+01 5.751257e+00 1.0
cibil_score 4269.0 5.998927e+02 1.724663e+02 297.0
residential_assets_value 4269.0 7.472617e+06 6.503637e+06 -100000.0
commercial_assets_value 4269.0 4.973969e+06 4.388288e+06 0.0
luxury_assets_value 4269.0 1.512677e+07 9.104654e+06 313517.0
bank_asset_value 4269.0 4.977307e+06 3.249207e+06 0.0

25% 50% 75% max
loan_id 1068.0 2135.0 3202.0 4269.0
no_of_dependents 1.0 2.0 4.0 6.0
income_annum 2659511.0 5081921.0 7503107.0 9942053.0
loan_amount 7702444.0 14547929.0 21473178.0 39492985.0
loan_term 6.0 11.0 16.0 21.0
cibil_score 453.0 600.0 748.0 901.0
residential_assets_value 2200000.0 5600000.0 11300000.0 29100000.0
commercial_assets_value 1344539.0 3694694.0 7640071.0 19380973.0
luxury_assets_value 7432747.0 14594092.0 21696591.0 39143904.0
bank_asset_value 2348272.0 4556261.0 7084046.0 14698452.0
```

Cell 7 - 7. Prepare features and target

```
Dropped ID-like columns: ['loan_id', 'income_annum', 'loan_amount', 'residential_assets_value',
'commercial_assets_value', 'luxury_assets_value', 'bank_asset_value']
Using 1200 sampled rows for model training speed.
Problem type: classification
Feature shape: (1200, 5)
Target unique values: 2
```

Cell 8 - 8. Train a baseline model

```
Training completed.
```

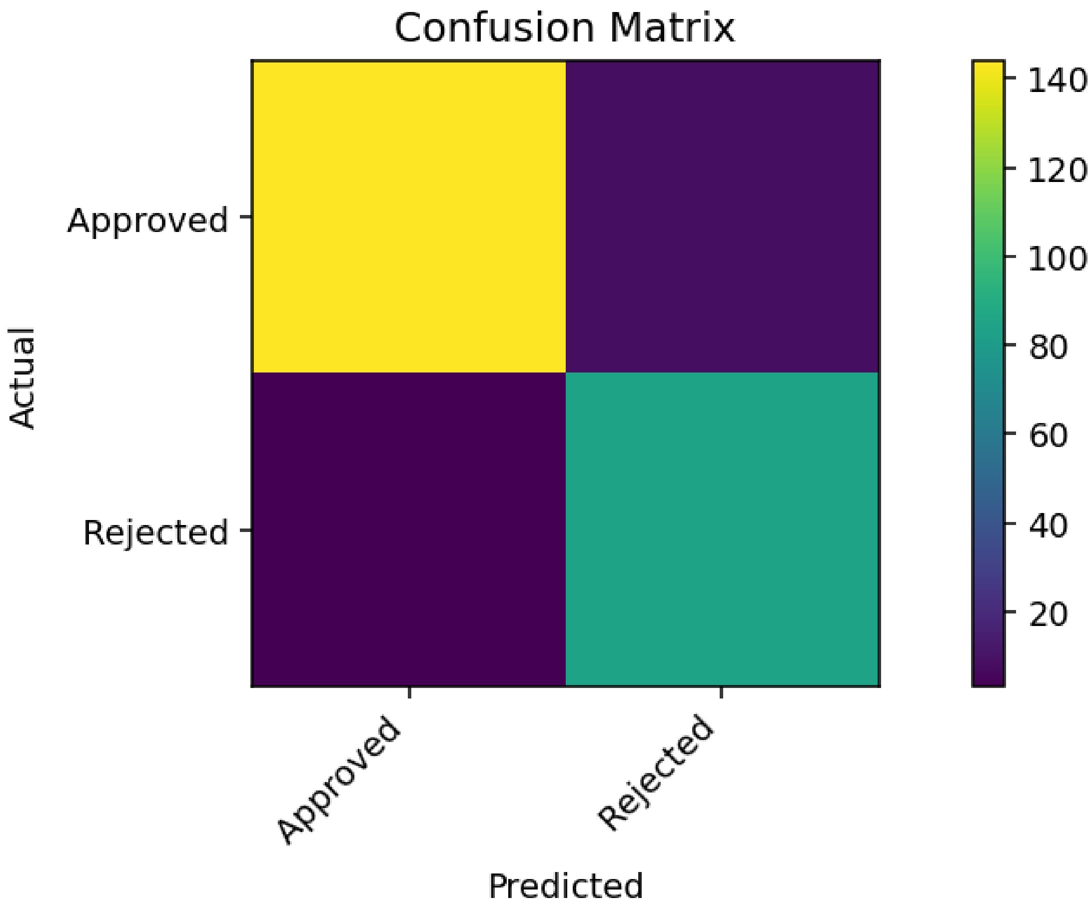
Cell 9 - 9. Evaluate the model

```
Accuracy: 0.9542

Classification report:
precision recall f1-score support

Approved 0.98 0.95 0.96 152
Rejected 0.91 0.97 0.94 88

accuracy 0.95 240
macro avg 0.95 0.96 0.95 240
weighted avg 0.96 0.95 0.95 240
```


Confusion Matrix
Approved
Rejected
Actual
Approved
Rejected
Predicted
140
120
100
80
60
40
20